

ТЗ: УНИКАЛИЗАТОР ВИДЕО

Модуль для окна экспорта Beatleap for Windows

ЦЕЛЬ МОДУЛЯ

При каждом экспорте видео автоматически применять набор незаметных для глаза, но уникальных для алгоритмов платформ изменений, которые делают каждый экспортированный файл уникальным. Это включает изменение метаданных, микроскопические визуальные вариации и технические параметры кодирования.

МЕСТО В ИНТЕРФЕЙСЕ

Уникализатор встраивается в **ExportModal** — модальное окно, которое открывается при нажатии кнопки «Сохранить».

Расположение: Отдельная секция в ExportModal между блоком выбора качества и кнопкой «Экспортировать».

Заголовок секции: Уникализация (текст `--text-secondary`, 12px uppercase, letter-spacing 1px)

Переключатель: Toggle-кнопка (как в iOS) справа от заголовка. По умолчанию — **включён**. Цвет активного состояния — `--accent-green`.

Раскрывающаяся панель: При включённом переключателе под заголовком разворачивается панель с настройками (анимация высоты 200ms ease). При выключенном — панель скрыта.

UI ПАНЕЛИ УНИКАЛИЗАТОРА

Панель содержит два блока: **Авто** (всегда применяется) и **Дополнительно** (пользователь выбирает).

Блок «Авто» (применяется всегда при включённом уникализаторе)

Текст-подсказка серым цветом `--text-secondary` 11px:

«Метаданные, хэш файла и технические параметры изменяются автоматически»

Блок «Дополнительно»

Чекбоксы с подписями (шрифт 13px, цвет `--text-primary`). Каждый чекбокс имеет подпись-описание 11px цвета `--text-secondary` под основным текстом.

Чекбокс	Описание под ним	По умолчанию
Цветовой сдвиг	Незаметное смещение цветового баланса	Включён
Зеркальный флип	Горизонтальный флип + обратный флип (незаметно)	Выключен
Шум	Добавить микро-зерно поверх видео	Включён
Скорость	Изменить скорость на $\pm 0.5-2\%$	Включён
Обрезка краёв	Обрезать 1–3px по краям и растянуть обратно	Включён
Аудио сдвиг	Сдвинуть аудиодорожку на 10–50ms	Включён

ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ

1. МЕТАДАННЫЕ (применяется всегда)

Реализовать в `src/utils/uniquazer.ts` , функция `randomizeMetadata(outputPath: string): Promise<void>` .

Использовать `fluent-ffmpeg` для перезаписи следующих метаданных через флаг `-metadata` :

Поле метаданных	Что записывать
<code>title</code>	Случайная строка 8–16 символов (латиница + цифры)
<code>comment</code>	UUID v4 (<code>crypto.randomUUID()</code>)
<code>creation_time</code>	Случайная дата в диапазоне: от 6 месяцев назад до вчера

encoder	Один из массива: ['Lavf58.76.100', 'Lavf59.16.100', 'Lavf60.3.100', 'Lavf57.83.100'] — выбирается случайно
major_brand	Один из: ['isom', 'mp42', 'M4V', 'qt'] — выбирается случайно

FFmpeg-команда:

Plain Text

```
ffmpeg -i input.mp4
  -metadata title="[random]"
  -metadata comment="[uuid]"
  -metadata creation_time="[random_date]"
  -metadata encoder="[random_encoder]"
  -codec copy
  output_meta.mp4
```

2. ХЭШ ФАЙЛА (применяется всегда)

После завершения FFmpeg-рендеринга дописать в конец файла случайный блок данных размером от 512 до 2048 байт (случайные байты через `crypto.getRandomValues`). Это изменяет MD5/SHA хэш файла без влияния на воспроизведение.

Реализовать в функции `appendRandomBytes(filePath: string): Promise<void>` :

TypeScript

```
const size = Math.floor(Math.random() * 1536) + 512; // 512-2048 байт
const randomBytes = crypto.getRandomValues(new Uint8Array(size));
await fs.appendFile(filePath, Buffer.from(randomBytes));
```

3. ЦВЕТОВОЙ СДВИГ (чекбокс «Цветовой сдвиг»)

FFmpeg `eq` фильтр с микроскопическими случайными значениями:

Plain Text

```
eq=brightness=[B]:contrast=[C]:saturation=[S]:gamma=[G]
```

Диапазоны случайных значений:

Параметр	Диапазон	Пример
brightness	от -0.03 до +0.03	0.017
contrast	от 0.97 до 1.03	1.012
saturation	от 0.96 до 1.04	0.983
gamma	от 0.97 до 1.03	1.008

Значения генерировать через `(Math.random() * range - range/2)` с точностью до 3 знаков после запятой.

4. ЗЕРКАЛЬНЫЙ ФЛИП (чекбокс «Зеркальный флип»)

Применяется только если пользователь включил чекбокс.

Стратегия: применить горизонтальный флип к первым 3 кадрам видео, затем вернуть обратно. Для алгоритмов это создаёт уникальный fingerprint начала видео, для зрителя — незаметно.

FFmpeg реализация через `select` + `hflip` для первых 3 кадров:

Plain Text

```
select='lt(n,3)',hflip,select='gte(n,3)'
```

Альтернативный подход (проще): применить `hflip` ко всему видео, затем сразу применить `hflip` обратно, но с разными параметрами кодирования первого и второго прохода. Итоговое видео выглядит идентично оригиналу, но имеет другой бинарный fingerprint.

5. ШУМ (чекбокс «Шум»)

FFmpeg `noise` фильтр:

Plain Text

```
noise=alls=[N]:allf=t+u
```

Где `[N]` — случайное значение от 1 до 4 (включительно). Значение 1–4 создаёт зерно, практически невидимое при нормальном просмотре, но меняющее хэш каждого кадра.

6. СКОРОСТЬ (чекбокс «Скорость»)

Изменить скорость воспроизведения на случайную величину от -0.5% до +2% (не включая 0%).

Диапазон: [0.995, 0.996, 0.997, 0.998, 0.999, 1.001, 1.002, 1.003, 1.004, 1.005, 1.010, 1.015, 1.020] — выбрать случайный элемент.

FFmpeg:

Plain Text

```
setpts=[SPEED]*PTS
```

Для аудио синхронно:

Plain Text

```
atempo=[AUDIO_SPEED]
```

Где `AUDIO_SPEED = 1 / SPEED` (компенсация, чтобы аудио не рассинхронизировалось).

7. ОБРЕЗКА КРАЁВ (чекбокс «Обрезка краёв»)

Обрезать случайное количество пикселей по каждому краю (1–3px), затем масштабировать обратно до исходного разрешения. Это меняет fingerprint каждого кадра.

Случайные значения для каждого края независимо:

TypeScript

```
const top    = Math.floor(Math.random() * 3) + 1; // 1-3
const bottom = Math.floor(Math.random() * 3) + 1;
const left   = Math.floor(Math.random() * 3) + 1;
const right  = Math.floor(Math.random() * 3) + 1;
```

FFmpeg:

Plain Text

```
crop=iw-[left+right]:ih-[top+bottom]:[left]:[top],scale=[original_width]:[original_height]
```

8. АУДИО СДВИГ (чекбокс «Аудио сдвиг»)

Сдвинуть аудиодорожку на случайное значение от 10 до 50 миллисекунд.

TypeScript

```
const delayMs = Math.floor(Math.random() * 41) + 10; // 10-50ms
```

FFmpeg через `adelay` :

Plain Text

```
adelay=[delayMs] | [delayMs]
```

ИНТЕГРАЦИЯ В FFMPEG PIPELINE

Все выбранные фильтры уникализатора добавляются в конец `vf` (video filter chain) и `af` (audio filter chain) в `ffmpegBuilder.ts` **после** применения пользовательских эффектов и фильтров из вкладок EDIT и FILTERS.

Порядок применения фильтров в финальном FFmpeg-вызове:

1. Пользовательские эффекты (из EDIT)
2. Пользовательский фильтр (из FILTERS)
3. Фильтры уникализатора (цветовой сдвиг, шум, обрезка, скорость)

После завершения FFmpeg-рендеринга:

1. Вызвать `randomizeMetadata(outputPath)` — перезаписать метаданные
2. Вызвать `appendRandomBytes(outputPath)` — добавить случайные байты

ПРОГРЕСС-БАР В ОКНЕ ЭКСПОРТА

Добавить в прогресс-бар дополнительный этап после основного рендеринга:

Этап	Текст в UI
0–85%	Рендеринг... X%
85–95%	Применяем уникализацию...
95–100%	Финализация файла...

ФАЙЛОВАЯ СТРУКТУРА

Новые файлы, которые нужно создать:

Plain Text

```
src/  
  utils/  
    unqualizer.ts      ← основной модуль (все функции)  
  components/  
    UnqualizerPanel.tsx ← UI-компонент панели в ExportModal  
  types/  
    unqualizer.d.ts    ← TypeScript-интерфейсы
```

src/types/unqualizer.d.ts :

TypeScript

```
export interface UnqualizerSettings {  
  enabled: boolean;  
  colorShift: boolean;  
  mirrorFlip: boolean;  
  noise: boolean;  
  speed: boolean;  
  cropEdges: boolean;  
  audioShift: boolean;  
}  
  
export interface UnqualizerResult {  
  appliedFilters: string[];  
  metadataChanges: Record<string, string>;  
  randomBytesAdded: number;  
}
```

Добавить в Zustand store:

TypeScript

```
unqualizerSettings: UnqualizerSettings;  
setUnqualizerSettings: (settings: Partial<UnqualizerSettings>) => void;
```

Дефолтное состояние:

TypeScript

```
unqualizerSettings: {  
  enabled: true,  
  colorShift: true,
```

```
mirrorFlip: false,  
noise: true,  
speed: true,  
cropEdges: true,  
audioShift: true,  
}
```

ACCEPTANCE CRITERIA

Критерий	Проверка
Панель уникализатора видна в ExportModal	Визуально
Toggle включает/выключает панель настроек	Клик на toggle
Каждый чекбокс сохраняется в store	Проверить DevTools
Два экспорта одного видео дают разные MD5-хэши	<code>certutil -hashfile file.mp4 MD5</code>
Два экспорта дают разные метаданные	<code>ffprobe -v quiet -print_format json -show_format file.mp4</code>
Цветовой сдвиг незаметен при просмотре	Визуальный просмотр
Шум незаметен при просмотре	Визуальный просмотр
Обрезка краёв незаметна при просмотре	Визуальный просмотр
Скорость незаметна при просмотре	Визуальный просмотр
Прогресс-бар показывает этап «Применяем уникализацию...»	Визуально при экспорте
Экспорт завершается без ошибок при всех включённых опциях	Тест экспорта